



Real World Planning and Acting

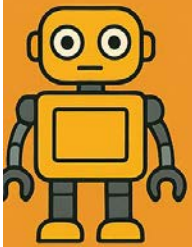
CS4246/CS5446

Reinforcement Learning and Sequential Decision Making

Version 4.0

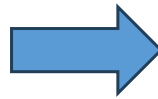
Types of AI Agents

1980s
Symbolic
AI agent



Block A is on block B
I want A on the table.
must first clear A, then
pick it up and move it:"

Logic rules,
symbolic planning



2020s
LLM agent



Let's reason
step by step.

"Let's reason step by step:
If the user wants a restau-
rant, I should find optio-
then suggest reviews..."

Language-based reasoning,
pretrained models

Example: Air Cargo Transport Planning



Init(*At*(*C*₁, *SFO*) $\wedge$ *At*(*C*₂, *SIN*) $\wedge$ *At*(*P*₁, *SFO*) $\wedge$ *At*(*P*₂, *SIN*)
 $\wedge$ *Cargo*(*C*₁) $\wedge$ *Cargo*(*C*₂) $\wedge$ *Plane*(*P*₁) $\wedge$ *Plane*(*P*₂)
 $\wedge$ *Airport* (*SIN*) $\wedge$ *Airport*(*SFO*))
Goal(*At*(*C*₁, *SIN*) $\wedge$ *At*(*C*₂, *SFO*))

Action(*Load*(*c*, *p*, *a*),
 PRECOND: *At*(*c*, *a*) $\wedge$ *At*(*p*, *a*) $\wedge$ *Cargo*(*c*) $\wedge$ *Plane*(*p*) $\wedge$ *Airport*(*a*)
 EFFECT: $\neg$ *At*(*c*, *a*) $\wedge$ *In*(*c*, *p*))
Action(*Unload*(*c*, *p*, *a*),
 PRECOND: *In*(*c*, *p*) $\wedge$ *At*(*p*, *a*) $\wedge$ *Cargo*(*c*) $\wedge$ *Plane*(*p*) $\wedge$ *Airport*(*a*)
 EFFECT: *At*(*c*, *a*) $\wedge$ $\neg$ *In*(*c*, *p*))
Action(*Fly*(*p*, *from*, *to*),
 PRECOND: *At*(*p*, *from*) $\wedge$ *Plane*(*p*) $\wedge$ *Airport*(*from*) $\wedge$ *Airport*(*to*)
 EFFECT: $\neg$ *At*(*p*, *from*) $\wedge$ *At*(*p*, *to*))

Planning Domain

Planning Problem

Adapted from source: RN Figure 11.1



Example: Planning Domain in PDDL

```
(define (domain air-cargo)
  (:requirements :strips :typing)
  (:types plane airport cargo)
  (:predicates
    (At ?x - (either plane cargo) ?a - airport)
    (In ?c - cargo ?p - plane)
    (Cargo ?c - cargo)
    (Plane ?p - plane)
    (Airport ?a - airport))
  (:action Load
    :parameters (?c - cargo ?p - plane ?a - airport)
    :precondition (and (At ?c ?a) (At ?p ?a) (Cargo ?c) (Plane ?p) (Airport ?a))
    :effect (and (not (At ?c ?a)) (In ?c ?p)))
  (:action Unload
    :parameters (?c - cargo ?p - plane ?a - airport)
    :precondition (and (In ?c ?p) (At ?p ?a) (Cargo ?c) (Plane ?p) (Airport ?a))
    :effect (and (At ?c ?a) (not (In ?c ?p))))
  (:action Fly
    :parameters (?p - plane ?from - airport ?to - airport)
    :precondition (and (At ?p ?from) (Plane ?p) (Airport ?from) (Airport ?to))
    :effect (and (not (At ?p ?from)) (At ?p ?to))))
```



Example: Planning Problem in PDDL

```
(define (problem air-cargo-problem)
  (:domain air-cargo)

  (:objects
    C1 C2 - cargo
    P1 P2 - plane
    SFO SIN - airport)

  (:init
    (At C1 SFO)
    (At C2 SIN)
    (At P1 SFO)
    (At P2 SIN)
    (Cargo C1)
    (Cargo C2)
    (Plane P1)
    (Plane P2)
    (Airport SFO)
    (Airport SIN))

  (:goal
    (and (At C1 SIN) (At C2 SFO))))
```



A Possible Plan

- A solution –
- Some caveats
 - What to do with spurious actions like *Fly*(P_1 , *SIN*, *SIN*)?



Classical Planning Summary

- Goal-directed planning
 - STRIPS, PDDL
- Search-based approaches:
 - Forward (progression)
 - Backward (regression)
- SAT-based approaches:
 - Encode as SAT (CNF)
 - Solved by SATPlan and other SAT planners
- Other approaches
 - Planning graph – encodes constraints, derives heuristics
 - Situation calculus – FOL representation of actions
 - CSP formulation – bounded planning problems
 - Partial-order planning – plans as graphs; supports parallelism and explainability



From Classical to Modern Planning

- Classical planning remains highly relevant
 - Crucial in robotics, logistics, space mission planning, game AI, and multi-agent coordination, etc.
- Scaled via:
 - Heuristics from relaxed models and abstractions
 - Hierarchical decomposition (HTNs, HLAs)
 - Emerging LLM-assisted methods for PDDL model generation, subgoal planning, and heuristic learning



Topics

- Enhance Efficiency
 - Heuristics for planning
 - Problem relaxation & admissible heuristics
 - Domain-independent approaches
- Reduce Complexity
 - State abstraction and Goal decomposition
 - Hierarchical planning
 - Hierarchical Task Networks (HTN)
 - High-level actions (HLAs)
- Scale with LLM-Assisted Planning
 - Planning model (PDDL) creation & refinement
 - Task decomposition + subgoal planning
 - Heuristic generation



Heuristics for Search-Based Planning

Enhancing efficiency

Heuristics for Classical Planning Paradigms

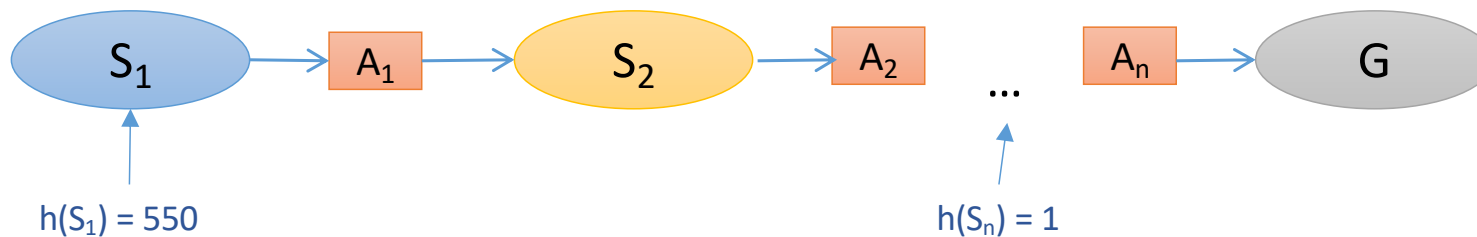
- Why use heuristics?
 - Reduce search space
 - Prune irrelevant branches
 - Find solutions faster
- Search-based planning (A*, GBFS, etc.)
 - Explicit heuristic function $h(s)$
 - Estimates cost from state $\rightarrow$ goal
 - Guides node expansion to reduce search
- Logical inference planning (SAT-based, CSP, etc.)
 - No explicit heuristic function $h(s)$
 - Relies on solver-internal heuristics:
 - Variable / clause selection
 - Branching & propagation strategies
 - Guides proof/search order indirectly

Heuristics are:
explicit in search, implicit in
inference-based methods.

Planning as Search Overview

- Definitions

- States (nodes): Represent world configurations.
- Actions (edges): Change state.
- Goal: find a path from initial state to goal state
- **Heuristic** $h(s)$: Estimates cost from state s to goal g



Improving Efficiency in Search-Based Planning

- **Admissible heuristics**

- Never overestimate → guarantee optimality (with optimal search, e.g., A^*)

- **Domain-independent heuristics**

- From relaxed problems → derive optimistic estimates

E.g., ignore some action conditions and/or effects

- **Domain-independent pruning**

- Reduce redundant or less promising branches in search space

- **Domain-specific heuristics**

- Tailored, (human-)designed for a given domain

Planning as Path Search

- Example: 8-Puzzle

- Game objective:

- Rearrange eight numbered tiles on a 3×3 board from a given start state to a specified goal state.

Initial

2		3
1	8	4
7	6	5



Goal

1	2	3
8		4
7	6	5

Action(Slide(t, s_1, s_2))

Precond: $On(t, s_1) \wedge Tile(t) \wedge Blank(s_2) \wedge Adjacent(s_1, s_2)$

Effect: $On(t, s_2) \wedge Blank(s_1) \wedge \neg On(t, s_1) \wedge \neg Blank(s_2)$

Example: 8-Puzzle as Planning

- States

e.g., $On("2", \langle 3, 1 \rangle)$

- A (state description) describes the position of each tile in the grid

- Actions or operators

- Tiles **slide** left, right, up, or down

- Goal test

- The tiles match the target arrangement.

- Path cost

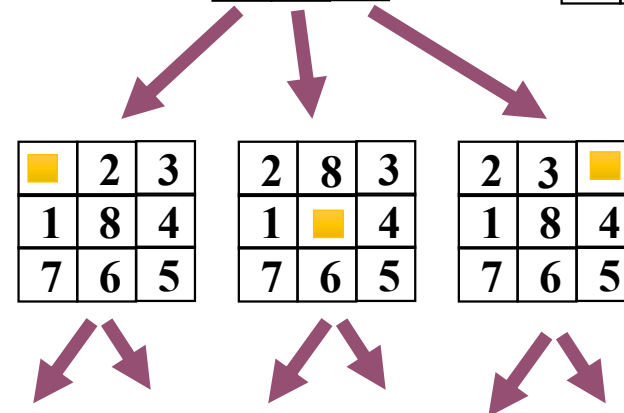
- Each move adds 1 to the path length

Initial

2	■	3
1	8	4
7	6	5

Goal

1	2	3
8	■	4
7	6	5



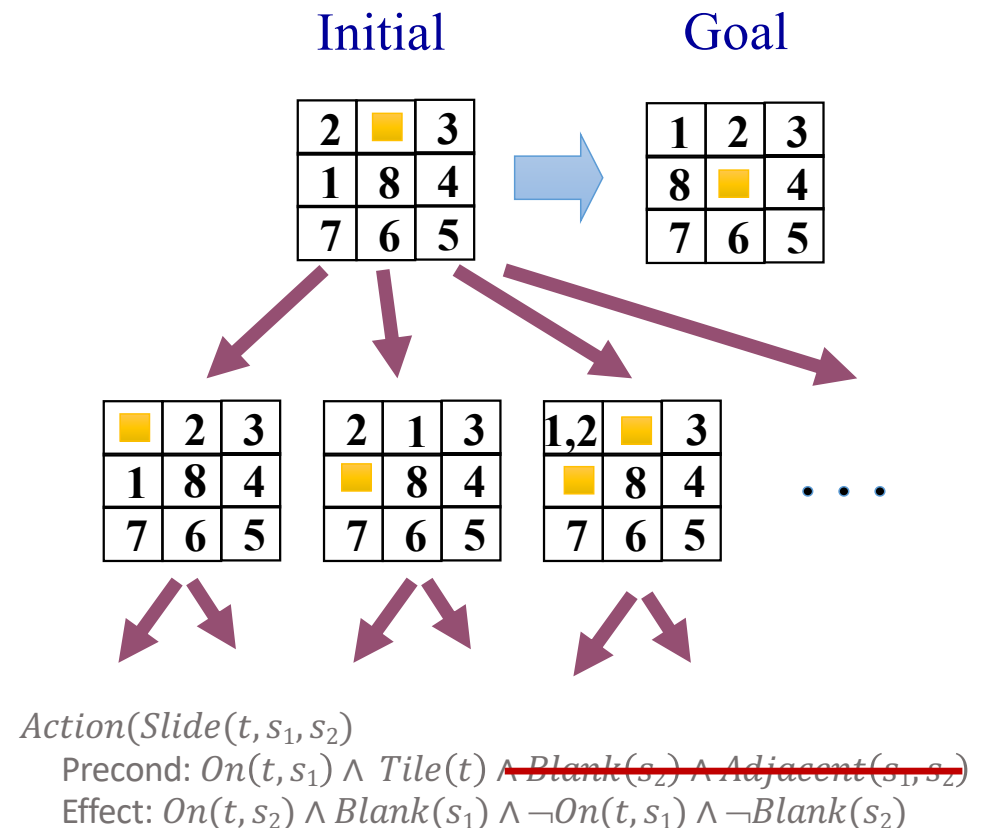
$Action(Slide(t, s_1, s_2))$

Precond: $On(t, s_1) \wedge Tile(t) \wedge Blank(s_2) \wedge Adjacent(s_1, s_2)$

Effect: $On(t, s_2) \wedge Blank(s_1) \wedge \neg On(t, s_1) \wedge \neg Blank(s_2)$

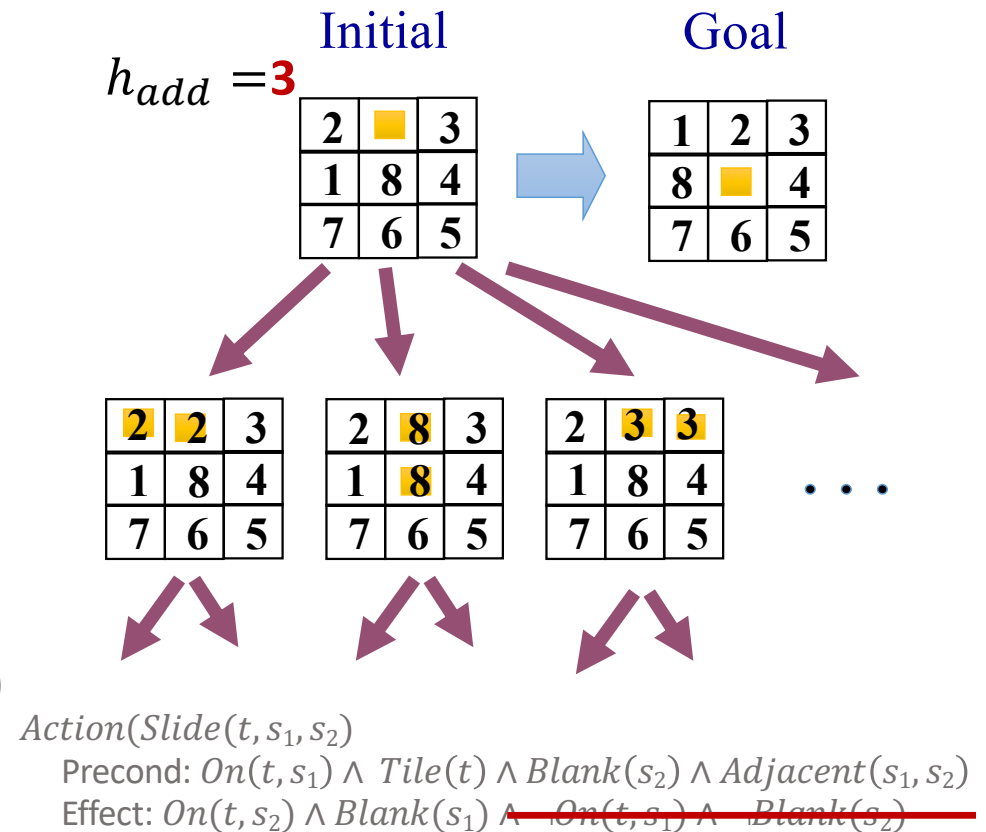
Ignore Selected Preconditions Heuristic

- Remove Preconditions
 - $Blank(s_2) \wedge Adjacent(s_1, s_2)$
- What is the heuristic?
 - No. of misplaced tiles heuristic
- What are the estimates?
- Challenge:
 - Unclear which preconditions can be selectively ignored in general



Ignore Delete Effects Heuristic

- Ignore delete effects
 - $\neg On(t, s_1) \wedge \neg Blank(s_2)$
- Manhattan distances: (min over copies)
 - E.g. Tile 1 = 1 Tile 2 = 1 Tile 8 = 1
 - Others = 0
- (Domain-independent) Heuristics:
 - $h_{max} : \max\{1,1,1\} = 1$
(admissible, weak)
 - $h_{add} : 1 + 1 + 1 = 3$
(more informative, not always admissible)
 - $h_{FF} : ?$
- What are the estimates?





Notes: h_{FF} : The Fast Forward Heuristic

- How it works (Hoffmann & Nebel, 2001)
 - **Ignore delete effects** → build a relaxed problem where facts never become false.
 - Compute relaxed plan → greedily select actions to achieve remaining goals.
 - Heuristic value → $h_{FF}(s)$ = number of actions in this relaxed plan.
- Key properties
 - Highly informed → more accurate than simpler heuristics like h_{max} .
 - Inadmissible → may *overestimate* cost because greedy extraction can give longer-than-optimal relaxed plans.
 - Effective in practice → widely used in satisficing planning (e.g. greedy best-first search).

Summary: Improving Efficiency

How to find preferred action(s)?

- Relaxation-based heuristics
 - Provide optimistic lower bounds; admissible heuristics ensure optimality
- Limitations:
 - Relaxed problems remain NP-hard;
 - Trade-off informativeness vs admissibility;
 - Plans may be redundant or costly to compute
- Key points
 - Heuristics guide search; pruning reduces it.
 - Combined (e.g., Fast Downward), they make planning more scalable and often optimal.
- Domain-independent pruning
 - **Symmetry reduction**: skip equivalent branches (e.g., Towers of Hanoi)
 - **Forward pruning**: prefer promising actions, may miss optimality (e.g., Fast Downward)
 - **Serializable subgoals**: plan goals in sequence, avoid undoing earlier ones (e.g., Sussman anomaly)



State Abstraction and Goal Decomposition

Reducing complexity



State Abstraction for Simplicity

- Key idea
 - Group states into abstract states → search in smaller space, then refine
- Steps
 - Define abstraction
 - Search abstract space
 - Map back to original space
- Challenge
 - Abstractions must reduce cost while preserving useful structure
- Examples:
 - Pattern databases: high setup cost but reusable across problems
 - Maze abstraction: rooms as states, refined to cell-level moves



Example: Air Cargo Transportation

- **Original problem:**
 - 10 airports, 50 planes and 200 cargos
 - Total no. of states =
- **Assumption:**
 - All cargos are just in 5 airports, instead of 10, and all cargos in the same airport have the same destination.
- **Reformulation:**
 - Drop all the irrelevant At fluents (What are the relevant ones?)
 - Total no. of states =
- **Solution:**
 - Shorter than that for original problem (admissible heuristic)
 - Extensible by adding relevant actions

Goal Decomposition for Efficiency

- Key idea

Key strategy in defining heuristics

- Break goal G into subgoals $G_1, \dots, G_n$
- For each subgoal G_i , compute a plan P_i with cost $Cost(P_i)$
- Use these costs to estimate the overall plan cost

- Heuristic rules

- $\text{Max}(\text{Cost}(P_i)) \rightarrow$ admissible, but weak
- $\text{Sum}(\text{Cost}(P_i)) \rightarrow$ admissible if subgoals are independent, more informative

- Interactions:

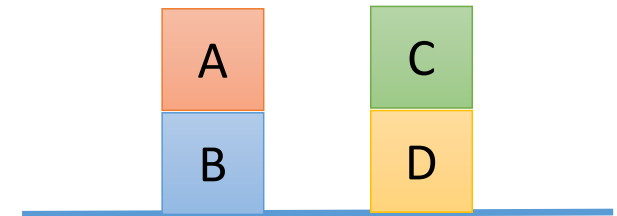
What if there is subgoal dependence?

- Positive $\rightarrow$ one action helps multiple subgoals $\rightarrow$ may overestimate
- Negative $\rightarrow$ one action undoes another $\rightarrow$ may underestimate

Example



- Initial State: All four blocks A, B, C and D are on the table.
- Goal G: Put block A on block B and block C on block D
 - Subgoal G_1 : A on B $\rightarrow$ Cost(P_1) = 2 steps (pickup and put)
 - Subgoal G_2 : C on D $\rightarrow$ Cost(P_2) = 2 steps
 - Subgoals are independent (plans do not interfere)



- Heuristic Estimates:
 - Max(Cost) =
 - Sum(Cost) =
- Why Sum is Better:
 - In the actual problem, both subgoals must be achieved, so the true cost is 4 steps
 - Max underestimates badly here
 - Sum matches the true cost (and is still admissible under independence)



Abstraction-Based Approaches

- **State Abstraction**
 - Group states into abstract ones → smaller search space
 - Example: Air Cargo with fewer airports / grouped cargos
 - **Benefit:** improves efficiency, admissible heuristics possible
- **Goal Decomposition**
 - Split complex goal into subgoals, estimate costs separately
 - Example: Blocksworld, A on B and C on D
 - **Benefit:** more informative heuristics, especially under independence
- **Key Insight**
- **Both simplify planning:**
 - State abstraction → fewer states to explore
 - Goal decomposition → clearer, stronger heuristic estimates



Hierarchical Planning and Acting

Manage complexity



Examples

- Example 1:

How to go to CS4246/5446 lecture from home?

- Solution: Go to COM2 from home, find lecture hall
- Solution: take MRT, change to internal bus, get off, find LT16.
- Solution: Switch on laptop, launch Zoom, join lecture

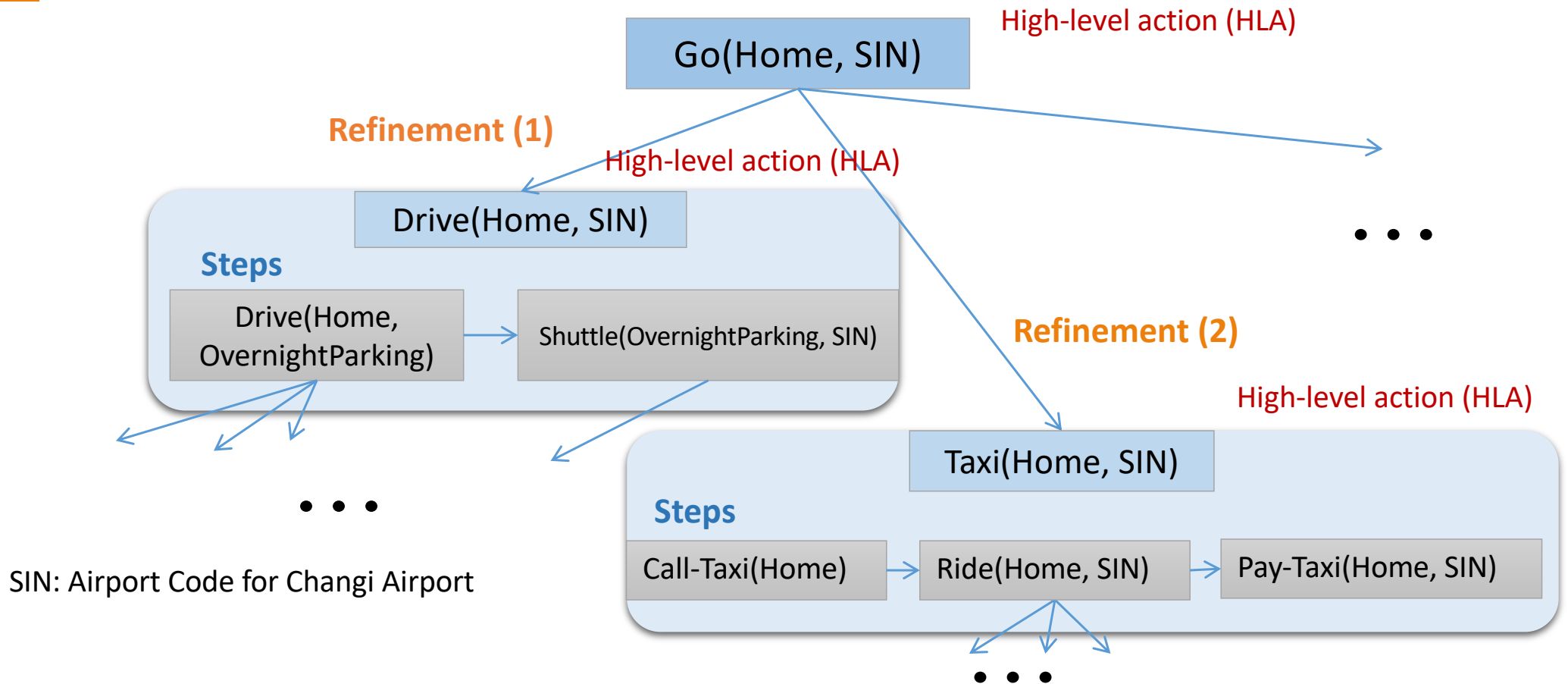
- Example 2:

How to land on the Jezero Crater of Mars from X space station?

- Example 3:

How to get from home to Changi airport?

Example: Going to Changi Airport



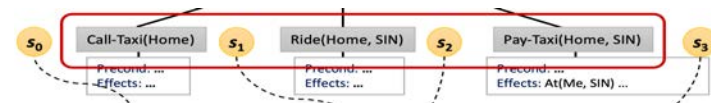


Managing Complexity in Planning

- Hierarchical decomposition:
 - Break tasks into smaller subtasks
 - Handle only a few at each level to reduce computation
- Deferred planning:
 - Plan partly in advance
 - Keep some actions abstract until execution

Hierarchical Task Networks (HTNs)

- Hierarchical decomposition planning
 - Built on **High-Level Actions (HLAs)**
 - Assumptions: full observability, determinism, primitive actions with preconditions/effects
 - Applications: planning, problem-solving, decision-making
- High-Level Actions (HLAs)
 - Refinable into sub-HLAs or primitive actions (recursive)
 - Encode **procedural knowledge**, support **multi-level planning**
 - HLA achieves a goal if at least one refinement succeeds (agent chooses)
 - Different from nondeterminism: choice is deliberate, not uncertain
 - Example: $\text{Go}(\text{Home}, \text{SIN}) \rightarrow \text{Drive} / \text{Taxi} / \text{MRT} \rightarrow \text{primitive steps}$





HTN Planning

- Main idea
 - Start with top-level task/action, refine until primitive actions only
- Algorithm
 - Select an HLA in the current plan
 - Replace with one refinement
 - Repeat until plan is primitive and satisfies goal
- Search strategies
 - Breadth-first (by refinement depth)
 - Depth-first, iterative deepening
 - Graph search

Example: Going to Changi Airport

Initial task:

Go(Home, SIN)

Refinement (1)

Drive(Home, SIN)

Precond: Have(Car)

Precondition fails

Refinement (2)

Taxi(Home, SIN)

Precond: Cash(30)

Precondition succeeds

I am at home, I have \$30,
I want to go to airport

Decomposition into
subtasks

Final state

Initial state

s_0

Call-Taxi(Home)

Precond: ...
Effects: ...

s_1

Ride(Home, SIN)

Precond: ...
Effects: ...

s_2

Pay-Taxi(Home, SIN)

Precond: ...
Effects: At(Me, SIN) ...

s_3

$s_0 = \{At(Me, Home), Cash(30)\}$

$s_1 = \{At(Me, Home), At(Taxi, Home), Cash(30)\}$

$s_2 = \{At(Me, SIN), At(Taxi, SIN), Cash(30)\}$

$s_3 = \{At(Me, SIN), At(Taxi, SIN), Cash(0)\}$

Adapted from: Dana Nau: Lecture slides for *Automated Planning*



Complexity Analysis

- Non-hierarchical planner:
 - Branching factor (actions) $b \rightarrow$ cost $O(b^d)$ for plan length d .
- HTN planner:
 - Each non-primitive action has r refinements into k actions $\rightarrow \approx r^{(d-1)/(k-1)}$ decompositions.
- Insight:
 - Small r , large $k \rightarrow$ compact HLA library (fewer refinements, longer sequences) $\rightarrow$ harder to design but more efficient.



Proving Plan Properties

Searching for Abstract Solutions

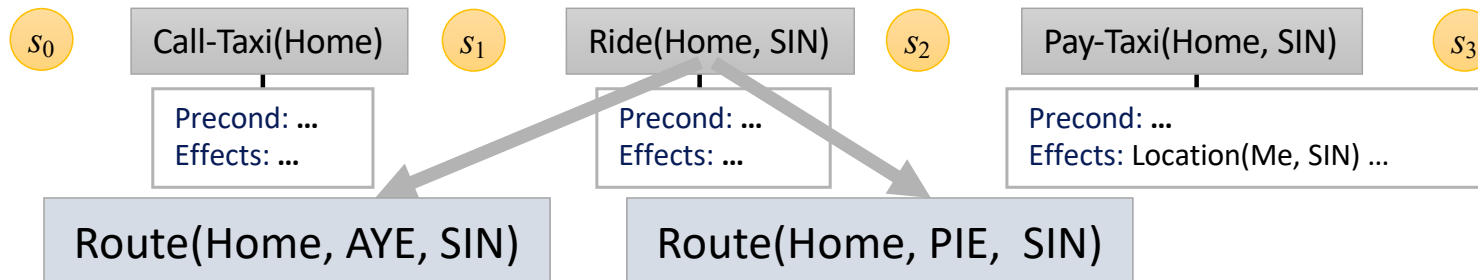
Motivation

- Example:

- Decide if a high-level plan achieves the goal (e.g., reach airport) without low-level details like route or terminal:
- [Call-Taxi(Home), Ride(Home, SIN), Pay-Taxi(SIN)]

Initial
state

Final
state

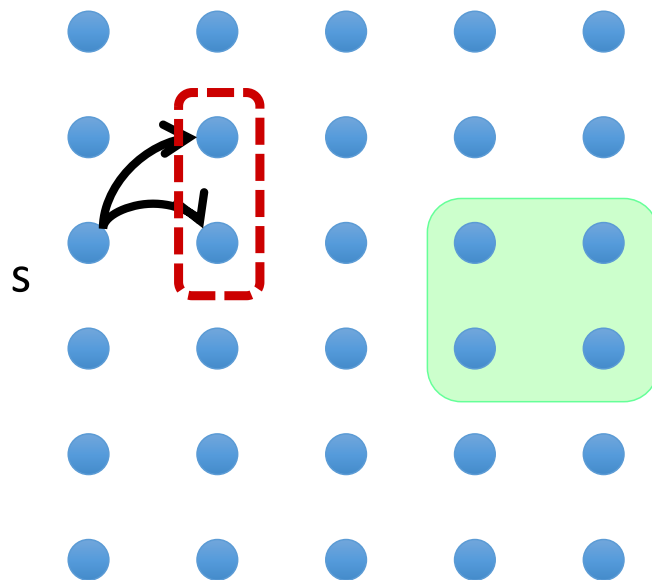




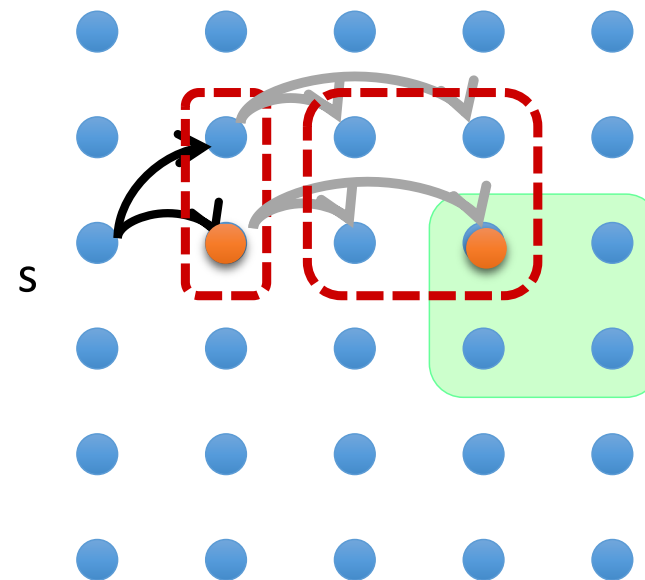
High-Level Planning with HLAs

- Goal
 - Decide if a high-level plan (HLA sequence) can achieve the goal without detailing all primitive steps
- Approach
 - Represent HLAs with preconditions & effects
 - Search over HLAs → commit when **reachable set** intersects goal states
 - Refine committed plan into primitive actions
- Benefit
 - Smaller search space → exponential reduction in complexity

Example: Reachable Sets



$REACH(s, [h_1])$



$REACH(s, [h_1, h_2])$

Source: RN: Fig 11.6

Reachable Sets

- Definition

- $REACH(s, h)$: all states reachable from state s by any implementation of HLA h

- Sequences

- $REACH(s, [h_1, h_2]) = \cup_{s' \in REACH(s, h_1)} REACH(s', h_2)$

- Key property

- **Downward refinement**: if a high-level plan reaches the goal, then at least one of its refinements (**into primitive actions**) will also achieve the goal.
- **Each HLA must be defined with complete and correct implementations**



Representing Reachable Set

- Notations:

- $\sim$ means possibly, if the agent chooses
- E.g., $\tilde{+}A$ means “possibly add A ”, i.e., either leave A unchanged or make it True

- Questions:

- What do $\simeq A$ and $\tilde{\pm}A$ mean?

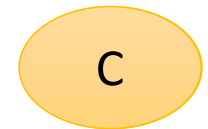
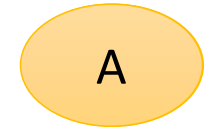
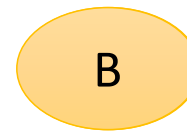
- Example

Go(Home, SIN) with two refinements

- *Drive(Home, SIN)* and *Taxi(Home, SIN)*
- Possibly delete *Cash* (if agent decides to take a taxi)
- So should have effect $\simeq \text{Cash}$

Example

- Consider:
 - Schemas for HLAs h_1 and h_2 :
 - Action (h_1 , Precond: $\neg A$, Effect: $A \wedge \simeq B$)
 - Action (h_2 , Precond: $\neg B$, Effect: $\tilde{+}A \wedge \tilde{\pm}C$)
- Meaning:
 - h_1 adds A and possibly delete B
 - h_2 possibly adds A and has full control over C
- Exercise:
 - If only B is true in the initial state and goal is $A \wedge C$
 - What sequence of HLAs will achieve the goal?
 - **$[h_1, h_2]$ achieves the goal**
 - Choose an implementation of h_1 that makes B false, then choose an implementation of h_2 that leaves A true and makes C true



Approximate Reachable Sets

- Why approximate?
 - HLAs may have infinitely many refinements
→ approximation keeps reasoning tractable and prunes early
- Types
 - **Optimistic** ($REACH^+$): overestimates → may miss failures
 - **Pessimistic** ($REACH^-$): underestimates → may miss successes
- General rule
 - $REACH^-(s, h) \subseteq REACH(s, h) \subseteq REACH^+(s, h)$.
- Key insight
 - If $REACH^+ \cap \text{Goal} = \emptyset \rightarrow \text{plan fails}$
 - If $REACH^- \cap \text{Goal} \neq \emptyset \rightarrow \text{plan succeeds}$
 - Otherwise → refine

Safe for pruning failures

Safe for certifying success

High-Level Planning with Approximate Descriptions

Reachable sets:

pessimistic

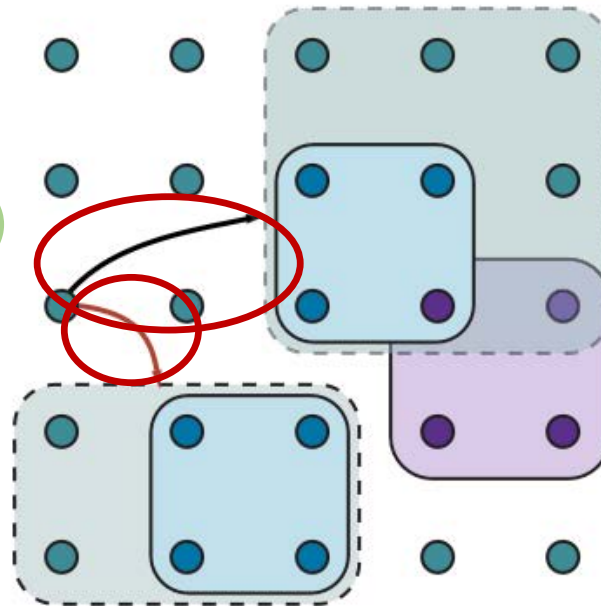
(solid lines, light blue)

optimistic

(dashed lines, light green)

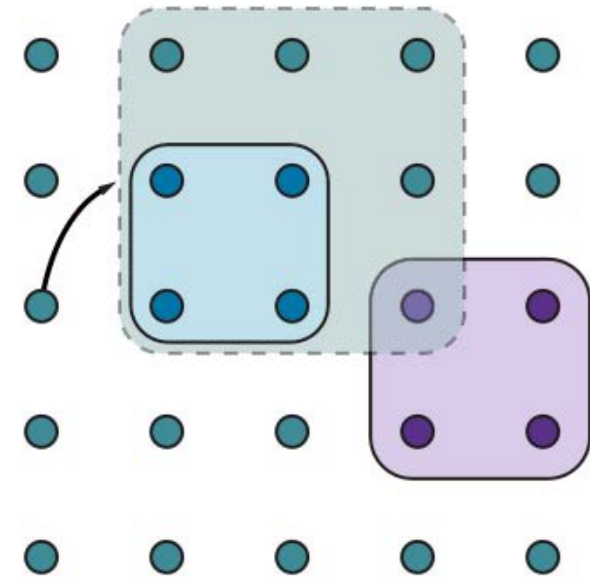
Definitely achieves goal

Needs refinement



Definitely NOT achieves goal

(a)



(b)

Source: RN: Fig 11.10



Summary

- Hierarchical decomposition:
 - Tasks $\rightarrow$ subtasks $\rightarrow$ primitives
- Abstraction & refinement
 - Reduce complexity, with downward refinement ensuring correctness
- Reachable sets
 - Guide correctness checks, with approximations ($REACH^+$, $REACH^-$)
- Pipeline:
 - Abstract Plan $\rightarrow$ Reachable Sets $\rightarrow$ Refinement $\rightarrow$ Primitive Plan
- HTN planning today:
 - Method libraries, learning & reuse, applied in planning, RL, robotics,
 - Now integrated with LLMs



LLM-Assisted Planning

Modeling

Task decomposition

Search optimization

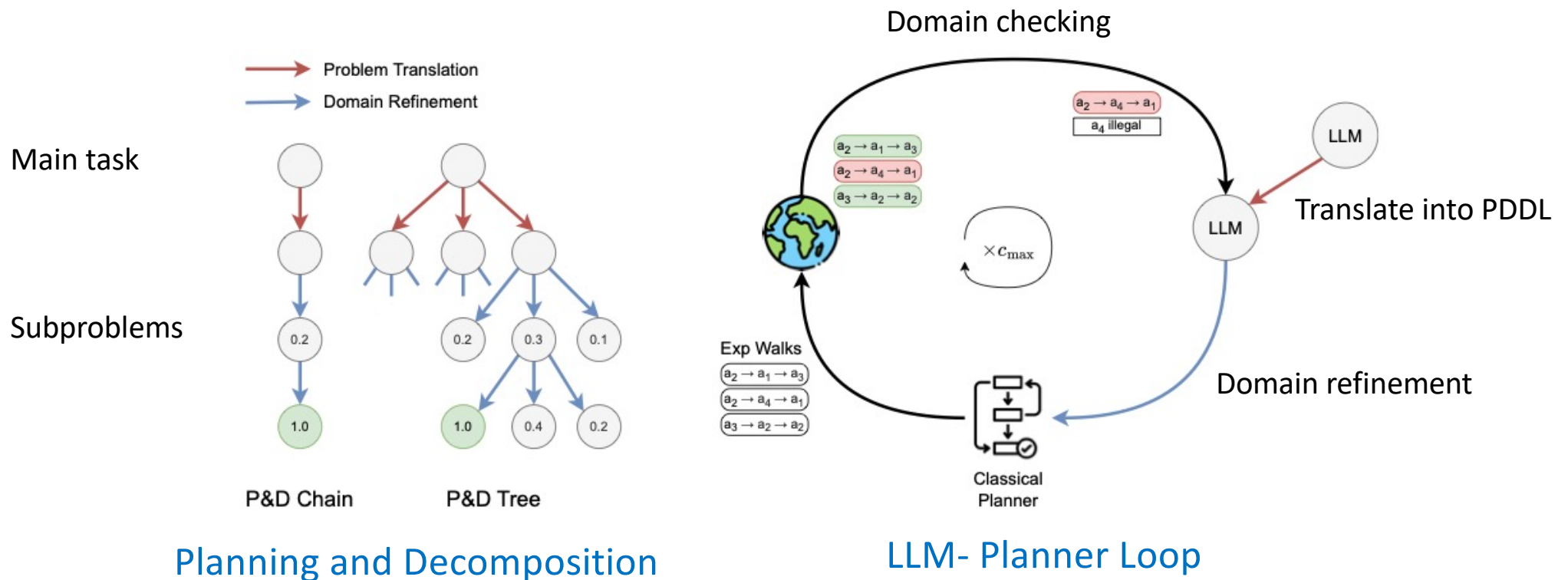
Real World Planning and Acting



LLM-Guided PDDL Creation and Refinement

- **Goal**
 - Automate PDDL domain/problem generation and refine models with feedback.
- **Approach**
 - LLM drafts initial PDDL from natural language goals domain and task descriptions
 - Evaluate generated PDDL in environment.
 - Iterative refinement via environment testing.
 - Planner executes final PDDL models.
- **Example**
 - *Mahdavi et al.*, NeurIPS 2024 [NeurIPS](#) | [GitHub](#)
 - LLM + feedback loop solves 66% tasks vs GPT-4's 29% intrinsic planning with CoT
 - Environment feedback greatly boosts planning success over LLM-only approaches
 - Planner: Fast Downward (with TextWorld modifications); plans validated with VAL.

LLM-assisted PDDL Creation





Task Decomposition & Subgoal Planning

- **Goal**
 - Combine LLM commonsense reasoning with symbolic planners for complex tasks.
- **Approach**
 - LLM decomposes high-level goals into subgoals.
 - Subgoals encoded in PDDL and solved by planners.
 - Complex subgoals handled by Monte Carlo Tree Search (MCTS) with LLM policy.
- **Example**
 - *Kwon et al.*, ICRA 2025 [ArXiv](#) | [Github](#)
 - Hybrid symbolic + MCTS planning reduces time and boosts scalability (reduces planning time while maintaining competitive success rates vs. baseline).
 - Planner: PDDL-based symbolic planner (Fast Downward as baseline) + MCTS-based LLM planner for complex subgoals.



Notes: Symbolic + MCTS with LLM

- Symbolic Planner (Baseline)

- For moderately complex subgoals, the system uses Fast Downward

- When to Switch

- If a sub-problem P_i is deemed too complex (e.g., by estimated Minimum Description Length or planning time), system switches from the symbolic planner to the Monte Carlo Tree Search (MCTS)-based LLM planner. [ArXiv](#)

- How it Works

- LLM (GPT-4) acts as rollout policy (L-Policy), sampling n_s candidate plans for P_i .
 - Candidates define a state tree T_i , with states as nodes and actions as edges.
 - MCTS explores this tree via selection, simulation, and backpropagation to find a valid plan that reaches the next desired subgoal S_{i+1}^* .



Heuristic Generation via LLMs

- Goal
 - Leverage LLMs to create domain-specific heuristics for classical planners.
- Approach
 - LLM synthesizes Python-coded heuristics from domain specs.
 - Heuristics plugged into classical planners to guide search.
- Example
 - *Corrêa et al., 2025* [ArXiv](#)
 - LLM-generated heuristics reduce explored states significantly
 - Approach outperforms standard domain-independent heuristics.
 - Planner: Pyperplan using Greedy Best-First Search (GBFS)



Generalized Planning in PDDL Domains with LLMs

- Goal
 - Leverage LLMs as generalized planners to learn reusable strategies across PDDL domains.
- Approach
 - LLM (GPT-4) prompted with PDDL domain descriptions and two training tasks.
 - LLM + Chain-of-Thought extracts domain rules and strategies.
 - LLM generates domain-specific Python programs for new tasks.
 - Automated debugging refines outputs via re-prompting.
- Example
 - *Silver et al., AAAI 2024* [AAAI](#)
 - No classical planner used — LLM generated programs directly, compared against classical baselines.
 - LLM (GPT-4) generalized across 7 PDDL domains with only 2 training tasks; often rivaling or surpassing classical baselines when automated debugging was applied.

Example: Generalized Planning in PDDL Domains with Pretrained Large Language Models (LLMs)

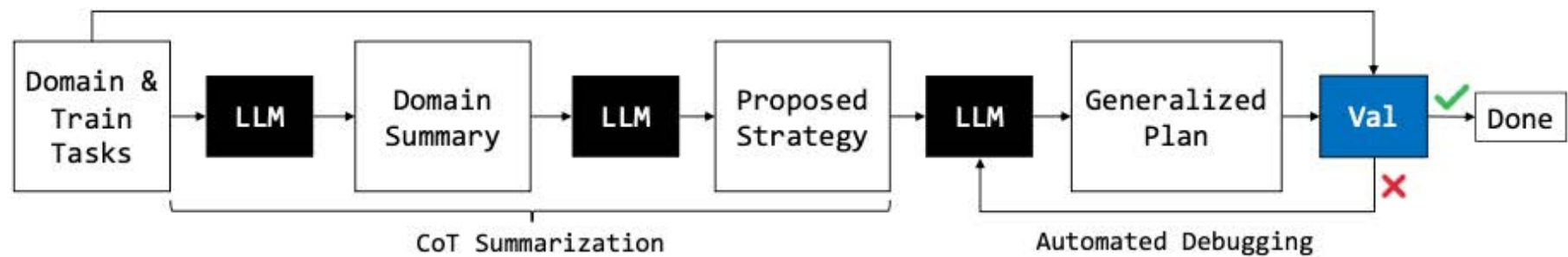


Figure 1: Overview of pipeline for generalized planning with pretrained LLMs. See text for details.

Source: Silver, T., S. Dan, K. Srinivas, J. B. Tenenbaum, L. Kaelbling and M. Katz (2024). Generalized planning in PDDL domains with pretrained large language models. Proceedings of the AAAI Conference on Artificial Intelligence.

<https://ojs.aaai.org/index.php/AAAI/article/view/30006>



From Classical to Modern Planning: Next Steps

- Classical planning remains highly relevant
 - Crucial in robotics, logistics, space mission planning, game AI, and multi-agent coordination, etc.
- • Scaled via:
 - Heuristics from relaxed models and abstractions
 - Hierarchical decomposition (HTNs, HLAs)
 - Emerging LLM-assisted methods for PDDL model generation, subgoal planning, and heuristic learning
- Yet real world environments involve uncertainty
 - Noisy sensors, incomplete information, and stochastic dynamics
 - Goals often involve trade-offs among preferences, risk, and utility
- Next Topics:
 - Modeling and planning and sequential decision making under uncertainty
 - → Utility Theory, Markov Decision Processes (MDPs), Reinforcement Learning



Homework

- Readings:

- Heuristics: RN: 11.3
- Hierarchical: RN: 11.4
- Summary: RN: 11.7

- Reviews:

- RN: 3.3, 3.5, 3.6 (Review of search and heuristics in problem solving)



References

- Classical and hierarchical planning systems:
 - (Journal articles publicly available online or through NUS Library e-Resources)
 - Hoffmann, J. and B. Nebel, The FF planning system: fast plan generation through heuristic search. J. Artif. Int. Res., 2001. 14(1): p. 253–302.
 - Helmert, M., The fast downward planning system. J. Artif. Int. Res., 2006. 26(1): p. 191–246.
 - Georgievski, I. and M. Aiello, HTN planning: Overview, comparison, and beyond. Artificial Intelligence, 2015. 222: p. 124-156.
 - Höller, D., et al., The PANDA Framework for Hierarchical Planning. KI - Künstliche Intelligenz, 2021. <https://rdcu.be/cn6Ra>
 - Höller, D., et al., HDDL: An Extension to PDDL for Expressing Hierarchical Planning Problems. Proceedings of the AAAI Conference on Artificial Intelligence, 2020. 34(06): p. 9883-9891. <https://cdn.aaai.org/ojs/6542/6542-13-9767-1-10-20200519.pdf>
 - Höller, D. (2026). Learning Heuristic Functions for HTN Planning. Proceedings of the AAAI Conference on Artificial Intelligence, 40(43), 36262–36270. <https://doi.org/10.1609/aaai.v40i43.40945>
- Classical planning in the Real World and recent trends: (excluding integration with large foundational models)
 - Neufeld, X., et al., Building a Planner: A Survey of Planning Systems Used in Commercial Video Games. IEEE Transactions on Games, 2019. 11(2): p. 91-108. <https://ieeexplore.ieee.org/document/8214211>
 - Sohrabi, S., AI Planning for Enterprise: Putting Theory Into Practice, in Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19. 2019, International Joint Conferences on Artificial Intelligence Organization. p. 6408–6410. <https://www.ijcai.org/proceedings/2019/0897.pdf>
 - Estlin, T., et al. Increased Mars Rover Autonomy using AI Planning, Scheduling and Execution. in Proceedings 2007 IEEE International Conference on Robotics and Automation. 2007. https://ai.jpl.nasa.gov/public/documents/papers/estlin_icra07_marsrover.pdf
 - Rabideau, G.; Wong, V.; Gaines, D.; Agrawal, J.; Chien, S.; Kuhn, S.; Fosse, E.; and Biehl, J. [Onboard Automated Scheduling for the Mars 2020 Rover](#). In Proceedings of the International Symposium on Artificial Intelligence, Robotics and Automation for Space, of i-SAIRAS'2020, Noordwijk, NL, 2020. European Space Agency https://ai.jpl.nasa.gov/public/documents/papers/M2020_OBP_i-SAIRAS2020_camera.pdf
 - Steinmetz, M., J. Hoffmann, A. Kovtunova and S. Borgwardt (2022). Classical planning with avoid conditions. Proceedings of the AAAI Conference on Artificial Intelligence 36(9): 9944-9952. 10.1609/aaai.v36i9.21232
 - Bercher, P. (2022). Hierarchical planning and reasoning about partially ordered plans — from theory to practice. AI Magazine 43(4): 353-364. 10.1002/aaai.12073 <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/22007>



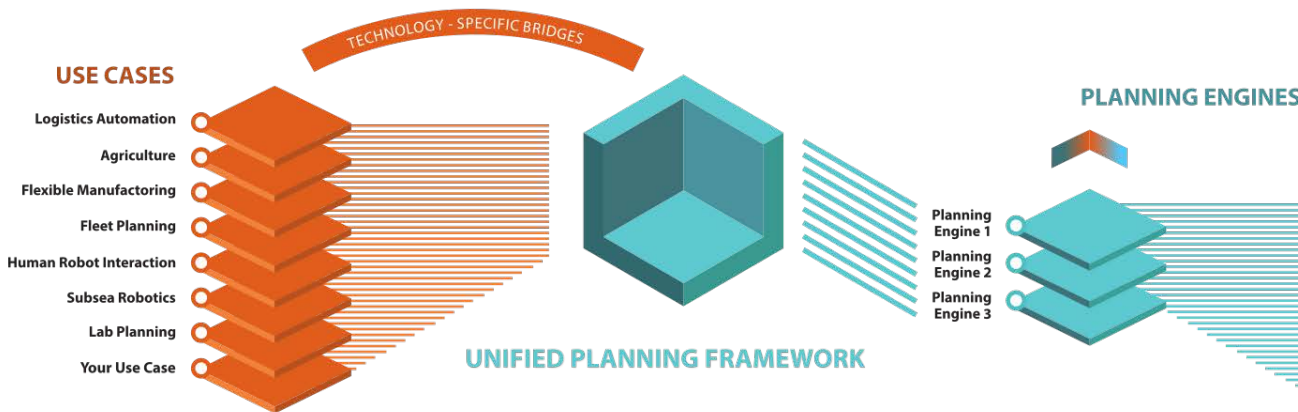
Appendix

Notes: Examples of LLM-Assisted Planning

Category	LLM + PDDL Integration	Key Contribution	Planner(s) Used	Reference
Model Creation	LLM-guided PDDL creation	Generate & refine PDDL models via feedback (Exploration Walk + VAL)	Fast Downward + VAL	NeurIPS 2024
Task Decomposition & Subgoal Planning	Task decomposition + subgoal planning	LLM decomposes tasks; symbolic planners / MCTS solve subgoals	Symbolic PDDL planner + MCTS	ICRA 2025
Heuristic Generation	Heuristic generation via LLM	Synthesize domain-specific heuristics to speed up search	Pyperplan with GBFS	ArXiv 2025
General Planning	Generalized planning via LLMs	LLM learns reusable planning strategies and generates solver-like code	None directly (strategy synthesis)	AAAI 2024

Notes: AIPlanning4EU Project

Making planning applicable
for everyone



Source: <https://www.aiplan4eu-project.eu>



https://youtu.be/nhO_F9SHEdM?si=k3_1_bb7P6ZbNmx

Classical Planning

The AIPlan4EU Unified Planning Library

CI-on-master passing codecov 85% docs passing pypi package 1.1.0

The unified_planning library makes it easy to formulate planning problems and to invoke automated planners.

- Define problems in a *simple, intuitive, and planner independent* way
- Solve your planning problems using one of the native solvers, or by using any PDDL planner
- Dump your problems in PDDL (or ANML) format
- Parse PDDL problem formulations
- Simplification, grounding, removal of conditional effects and many other transformations are available
- and more...

Usage

```
from unified_planning.shortcuts import *

x = Fluent("x")

a = InstantaneousAction("a")
a.add_precondition(Not(x))
a.add_effect(x, True)

problem = Problem("basic")
problem.add_fluent(x)
problem.add_action(a)
problem.set_initial_value(x, False)
problem.add_goal(x)

with OneshotPlanner(problem_kind=problem.kind) as planner:
    result = planner.solve(problem)
    if result.status in unified_planning.engines.results.POSITIVE_OUTCOMES:
        print(f'{planner.name} found this plan: {result.plan}')
    else:
        print("No plan found.")
```

<https://github.com/aiplan4eu/unified-planning>

Notes: The PANDA Framework

- PANDA – Planning and Acting in a Network Decomposition Architecture ([GitHub](#), [Uni Ulm](#))
 - Supports hierarchical planning with multiple solving techniques:
 - PANDApss – Heuristic plan space search (HTN + POCL).
 - PANDApro – Heuristic progression search.
 - PANDAtotSAT – Solves totally ordered HTN planning via SAT encoding.
- HDDL: Extension to PDDL for hierarchical planning
 - [AAAI 2020 paper](#)
 - Overview: [KI 2021 paper](#).
- PANDADealer – Totally ordered HTN planner ([GitHub](#))
 - Winner in HTN categories at IPC 2023 ([ICAPS 2023 Proc.](#)).
 - Builds on PANDA framework with optimizations for totally ordered tasks.
- Learning heuristic functions for HTN Planning
 - Holler – AAAI-2026
 - <https://doi.org/10.1609/aaai.v40i43.40945>

International Planning Competition 2023 HTN Tracks
IPC 2023 HTN Tracks

Results

The IPC 2023 has concluded and the results have been presented at ICAPS 2023 in Prague. The results presentation can be found [here](#). The full results tables can be found [here](#). The results website also contains links to CSV files of the results (and additionally coverage data).

You can download the raw log files of the planners and the per-instances scores for the TO Track and the PO Track.

The winners are:

Track	Winner	Runner-Up
Total-Order Satisficing	PandaDealer-agile-lama Conny Olt, Daniel Höller, Pascal Bercher	PANDAprø Å AO Daniel Höller
Total-Order Agile	PandaDealer-agile-1 and PandaDealer-agile-lama Conny Olt, Daniel Höller, Pascal Bercher	PANDAprø Å Im-cut Daniel Höller
Total-Order Optimal	PandaDealer-optimal Conny Olt, Daniel Höller, Pascal Bercher	PANDAprø Im-cut and PANDAprø DOF Daniel Höller
Partial-Order Satisficing	Grounded-Linear-Satisficing Ying Xian Wu, Conny Olt, Songtuan Lin, Pascal Bercher	PANDAprø Å AO and Im-cut Daniel Höller
Partial-Order Agile	Grounded-Linear-Agile Ying Xian Wu, Conny Olt, Songtuan Lin, Pascal Bercher	PANDAprø Å AO and Im-cut Daniel Höller
Partial-Order Optimal	PANDAprø Im-cut Daniel Höller	Aries Arthur Bit-Monnot

The full results tables can be found [here](#).

Source: <https://ipc2023-htn.github.io>